

ALGORITMOS E PROGRAMAÇÃO

Tutorial de Resolução

Segunda Lista de Exercícios — Comandos Condicionais em C

Este tutorial acompanha a aula de Comandos Condicionais. Ele traz, para cada um dos 21 exercícios da lista: o que o enunciado pede, as tabelas de referência já transcritas, o raciocínio antes do código, o programa completo e comentado, e os casos de teste que você deve rodar para ter certeza de que acertou.

Todos os programas deste material foram compilados com gcc -Wall -Wextra e executados com os casos de teste indicados.

Prof. William Malvezzi, MSc.

Como usar este tutorial

A pior forma de usar este material é ler a solução antes de tentar. A segunda pior é copiar o código sem entender por que ele está escrito daquele jeito. Sugestão de uso:

1. Leia o enunciado do exercício na lista original e tente resolver sozinho, no papel ou no computador, por pelo menos dez minutos.
2. Se travar, leia apenas a seção “Como pensar” do exercício correspondente aqui. Ela dá o caminho sem entregar o código.
3. Volte a tentar. Só depois compare com a seção “Solução”.
4. Rode os casos de teste indicados. Se algum der diferente, o problema está no seu código — e descobrir onde é a parte que mais ensina.

Uma observação sobre acentos

Os programas deste tutorial não usam acentos dentro de `printf`. Isso evita problemas de codificação de caracteres no terminal do Windows, que costumam confundir mais do que ajudar nesta fase do curso. Nos comentários e no texto, os acentos estão normais.

Parte 1 — O método de resolução

Quase todos os exercícios desta lista seguem a mesma receita. Vale investir cinco minutos entendendo o método: ele transforma vinte e um problemas diferentes em vinte e uma aplicações do mesmo procedimento.

Passo 1 — Separe entrada, processamento e saída

Leia o enunciado com uma caneta na mão e marque três coisas: o que o programa recebe do usuário, o que ele precisa calcular e o que ele deve mostrar. A lista de variáveis sai pronta dessa marcação.

Exemplo, no exercício 1: entrada são as três notas; o processamento é a média ponderada e o conceito; a saída são esses dois valores. Logo, precisamos de três variáveis reais para as notas, uma para a média e uma para o conceito.

Passo 2 — Transforme cada tabela em uma escada else-if

Toda tabela de faixas do tipo “de tanto até tanto, faça isso” vira diretamente uma sequência de else if. Cada linha da tabela é um ramo.

```
/* Tabela:  >= 8,00  -> A
            >= 7,00  -> B
            >= 6,00  -> C
vira:
*/

if (media >= 8.0)
    conceito = 'A';
else if (media >= 7.0)
    conceito = 'B';
else if (media >= 6.0)
    conceito = 'C';
```

Por que só o limite de baixo?

Quando o programa chega ao segundo teste, o primeiro já falhou — então ele já sabe que a média é menor que 8. Escrever `media >= 7.0 && media < 8.0` funciona, mas a segunda metade é redundante. Ordene as faixas da mais alta para a mais baixa e cada teste fica com uma comparação só.

Passo 3 — Escolha entre if e switch

A regra é curta:

- Se a decisão envolve faixas de valores, ou usa `<`, `>`, `<=` e `>=`, ou compara números reais: use `if / else if`.
- Se a decisão compara uma variável inteira (ou `char`) com valores exatos — códigos, opções de menu, números de mês: use `switch`.

Nesta lista, os exercícios 8 e 13 são menus e pedem `switch`. Os exercícios 12 e 21 têm códigos e também ficam elegantes com `switch`. Todos os demais que envolvem tabelas de faixa pedem `else-if`.

Passo 4 — Escreva o esqueleto antes das contas

Monte primeiro os if e else vazios, com as chaves no lugar. Só depois preencha o interior de cada ramo com as fórmulas. Isso evita o erro mais comum em exercícios longos: perder a estrutura no meio de uma sequência de cálculos.

Passo 5 — Teste as fronteiras, não o meio

Se a faixa vai até R\$ 500,00, não teste com 300. Teste com 499, 500 e 501. Praticamente todo erro de escada else-if se manifesta exatamente no valor de limite — e só ali.

Regra prática de teste

Para cada tabela do enunciado, monte uma lista com um valor de cada faixa mais os valores exatos de cada fronteira. Se a tabela tem três faixas, isso dá cerca de sete testes. Rodar os sete leva dois minutos e pega quase todos os erros possíveis.

Parte 2 — Referência rápida

Sintaxe dos comandos condicionais

```
/* 1) Condicional simples */
if (condição) {
    comandos;
}

/* 2) Condicional composta */
if (condição) {
    comandos se verdadeiro;
} else {
    comandos se falso;
}

/* 3) Escada de casos */
if (condição1) {
    ...
} else if (condição2) {
    ...
} else {
    ... /* tudo o que sobrou */
}

/* 4) Seleção por valor exato */
switch (variável_inteira) {
    case valor1:
        comandos;
        break;
    case valor2:
        comandos;
        break;
    default:
        comandos;
}
```

Operadores

Categoria	Operadores	Observação
Relacionais	> < >= <= == !=	Devolvem 1 (verdadeiro) ou 0 (falso).
Lógicos	&& !	&& exige as duas; basta uma; ! inverte.
Aritméticos	+ - * / %	% é o resto da divisão inteira.
Atribuição composta	+= -= *= /= %=	x += 5 é o mesmo que x = x + 5.

Precedência, da mais alta para a mais baixa: parênteses, depois !, depois * / %, depois + -, depois < <= > >=, depois == !=, depois &&, e por último ||. Na dúvida, use parênteses.

Funções de math.h

Função	O que faz	Exemplo
sqrt(x)	raiz quadrada	sqrt(16.0) devolve 4.0
pow(x, y)	x elevado a y	pow(2.0, 10) devolve 1024.0
fabs(x)	valor absoluto de um real	fabs(-7.5) devolve 7.5

Função	O que faz	Exemplo
ceil(x) / floor(x)	arredonda para cima / para baixo	ceil(2.1) devolve 3.0
log(x) / log10(x)	logaritmo natural / base 10	log10(100) devolve 2.0

Compilando com math.h

No Linux e no Mac é preciso ligar a biblioteca matemática na hora de compilar: `gcc programa.c -o programa -lm`. Se aparecer a mensagem “undefined reference to sqrt”, não é erro no seu código — é a falta do `-lm`. No Dev-C++ e em outras IDEs do Windows isso já vem configurado.

Os seis erros que mais aparecem

Erro	Como aparece	Correção
= no lugar de ==	if (a = 0) — o programa atribui em vez de comparar e a condição fica sempre falsa.	Use == para comparar. O = sozinho só serve para atribuir.
; depois do if	if (x > 0); — o bloco seguinte roda sempre, mesmo com x negativo.	Nunca coloque ponto-e-vírgula logo após o parêntese do if.
Chaves faltando	O if governa só a primeira linha; a segunda roda sempre, apesar da indentação.	Use chaves sempre, mesmo com um comando só.
Faixas fora de ordem	Numa escada else-if, o primeiro teste verdadeiro vence e os demais nem são avaliados.	Com >=, comece pela faixa mais alta. Com <=, pela mais baixa.
break esquecido	A execução escorre para o case seguinte e imprime coisas a mais.	Todo case precisa terminar com break (salvo agrupamento proposital).
Comparar reais com ==	Com float, um teste como altura <= 1.70 pode falhar exatamente em 1,70.	Use double e leia com %lf; nunca compare reais com ==.

Parte 3 — Os 21 exercícios resolvidos

Exercício 1 — Média ponderada e conceito

O que o enunciado pede

Receber três notas — trabalho de laboratório, avaliação semestral e exame final —, calcular a média ponderada segundo os pesos da tabela e mostrar a média junto com o conceito correspondente.

Pesos das notas

Nota	Peso
Trabalho de laboratório	2
Avaliação semestral	3
Exame final	5

Conceito por faixa de média

Média ponderada	Conceito
8,00 até 10,00	A
7,00 até 7,99	B
6,00 até 6,99	C
5,00 até 5,99	D
0,00 até 4,99	E

Como pensar

1. Média ponderada é a soma de cada nota multiplicada pelo seu peso, dividida pela soma dos pesos. Aqui os pesos somam $2 + 3 + 5 = 10$, então o divisor é 10.
2. Cuidado com a divisão: escreva 10.0 e não 10. Se todos os valores envolvidos forem inteiros, o C faz divisão inteira e joga fora a parte decimal. Como as notas são float, aqui não haveria problema, mas escrever 10.0 deixa a intenção explícita.
3. A tabela de conceitos é uma escada else-if clássica. Ordene do 8,0 para baixo e cada ramo precisa de uma comparação só.
4. O conceito é uma letra, então guarde-o em uma variável do tipo char e imprima com %c. Aspas simples para caractere ('A'), aspas duplas seriam texto.

Solução

```
/* Exercício 1 - Media ponderada e conceito */
#include <stdio.h>

int main() {
    float lab, sem, exame, media;
    char conceito;

    printf("Nota do trabalho de laboratorio (peso 2): ");
    scanf("%f", &lab);
    printf("Nota da avaliacao semestral (peso 3): ");
    scanf("%f", &sem);
    printf("Nota do exame final (peso 5): ");
    scanf("%f", &exame);
```

```

media = (lab * 2 + sem * 3 + exame * 5) / 10.0;

if (media >= 8.0)
    conceito = 'A';
else if (media >= 7.0)
    conceito = 'B';
else if (media >= 6.0)
    conceito = 'C';
else if (media >= 5.0)
    conceito = 'D';
else
    conceito = 'E';

printf("Media ponderada: %.2f\n", media);
printf("Conceito: %c\n", conceito);
return 0;
}

```

Teste você mesmo

Entrada	Saída esperada
10 10 10	Média 10,00 — conceito A
8 8 8	Média 8,00 — conceito A (fronteira)
7 7 7	Média 7,00 — conceito B (fronteira)
6 6 6	Média 6,00 — conceito C (fronteira)
5 5 5	Média 5,00 — conceito D (fronteira)
4 4 4	Média 4,00 — conceito E
10 5 2	Média 4,50 — conceito E (pesos diferentes fazem diferença)

Exercício 2 — Média aritmética, situação e nota de exame

O que o enunciado pede

Receber três notas, calcular a média aritmética e mostrar a mensagem correspondente. Para quem ficou de exame, calcular também a nota que precisa ser tirada no exame para ser aprovado, sabendo que a média mínima no exame é 6,00.

Situação por faixa de média

Média aritmética	Mensagem
0,00 até 2,99	Reprovado
3,00 até 6,99	Exame
7,00 até 10,00	Aprovado

Como pensar

1. A primeira parte é a mesma escada do exercício anterior, agora com três faixas em vez de cinco.
2. A parte que exige raciocínio é a nota do exame. A regra usual é que a média final seja a média entre a média do semestre e a nota do exame, e que essa média final precise chegar a 6,00.
3. Escrevendo isso: $(media + nota_exame) / 2 \geq 6,00$. Isolando a incógnita: $media + nota_exame \geq 12,00$, ou seja, $nota_exame \geq 12,00 - media$.
4. Esse cálculo só faz sentido para quem está de exame. Por isso ele fica dentro do ramo do meio da escada, e não solto no final do programa.

Onde os alunos escorregam

É comum ver o cálculo da nota do exame escrito fora do if, o que faz o programa imprimir uma “nota necessária” até para quem foi aprovado. O cálculo pertence ao ramo do exame — e é justamente para isso que servem as chaves.

Solução

```
/* Exercício 2 - Media aritmetica, situacao e nota de exame */
#include <stdio.h>

int main() {
    float n1, n2, n3, media, nota_exame;

    printf("Digite as tres notas: ");
    scanf("%f %f %f", &n1, &n2, &n3);

    media = (n1 + n2 + n3) / 3.0;
    printf("Media aritmetica: %.2f\n", media);

    if (media < 3.0) {
        printf("Reprovado\n");
    } else if (media < 7.0) {
        printf("Exame\n");
        /* media final = (media + nota_exame)/2 >= 6 => nota_exame >= 12 - media */
        nota_exame = 12.0 - media;
        printf("Nota necessaria no exame: %.2f\n", nota_exame);
    } else {
        printf("Aprovado\n");
    }
}
```

```
    return 0;  
}
```

Teste você mesmo

Entrada	Saída esperada
2 2 2	Média 2,00 — Reprovado
3 3 3	Média 3,00 — Exame, precisa de 9,00 (fronteira)
5 5 5	Média 5,00 — Exame, precisa de 7,00
6.99 6.99 6.99	Média 6,99 — Exame, precisa de 5,01
7 7 7	Média 7,00 — Aprovado (fronteira)
10 10 10	Média 10,00 — Aprovado

Exercício 3 — O maior entre dois números

O que o enunciado pede

Receber dois números e mostrar o maior.

Como pensar

1. É o if/else mais simples que existe. A única decisão de projeto é o que fazer quando os dois números são iguais.
2. Se você escrever apenas if (a > b) ... else ..., o caso de igualdade cai no else e o programa anuncia b como maior — o que é tecnicamente falso. Tratar a igualdade em um terceiro ramo deixa o programa honesto.
3. Note que a solução usa float e não int: o enunciado diz “números”, sem restringir a inteiros.

Solução

```
/* Exercício 3 - Maior entre dois numeros */
#include <stdio.h>

int main() {
    float a, b;

    printf("Digite dois numeros: ");
    scanf("%f %f", &a, &b);

    if (a > b)
        printf("O maior e: %.2f\n", a);
    else if (b > a)
        printf("O maior e: %.2f\n", b);
    else
        printf("Os numeros sao iguais: %.2f\n", a);

    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
8 3	O maior é 8,00
3 8	O maior é 8,00
5 5	Os números são iguais: 5,00
-2 -7	O maior é -2,00 (negativos também funcionam)

Exercício 4 — Três números em ordem crescente

O que o enunciado pede

Receber três números e mostrá-los em ordem crescente.

Como pensar

1. A tentação é testar as seis ordens possíveis com uma sequência enorme de if. Funciona, mas não escala: com quatro números seriam vinte e quatro casos.
2. A técnica melhor é ordenar por trocas. Compare os valores dois a dois e troque-os de lugar quando estiverem fora de ordem, até que todos estejam arrumados.
3. Para trocar o conteúdo de duas variáveis é obrigatório usar uma terceira, aqui chamada aux. Se você escrever `a = b` seguido de `b = a`, o valor original de `a` já foi perdido na primeira linha e as duas variáveis acabam iguais.
4. São necessárias exatamente três comparações: a primeira garante $a \leq b$; a segunda leva o maior de todos para `c`, mas pode desarrumar o par `a, b`; a terceira conserta esse par.

A troca de valores, passo a passo

`aux = a` guarda o valor de `a` em um lugar seguro. `a = b` copia `b` para `a`. `b = aux` recupera o valor antigo de `a` e coloca em `b`. Ao final, os dois valores trocaram de lugar e nada se perdeu.

Solução

```
/* Exercício 4 - Tres numeros em ordem crescente (ordenacao por trocas) */
#include <stdio.h>

int main() {
    float a, b, c, aux;

    printf("Digite tres numeros: ");
    scanf("%f %f %f", &a, &b, &c);

    /* Garante a <= b */
    if (a > b) { aux = a; a = b; b = aux; }
    /* Garante b <= c (o maior vai para c) */
    if (b > c) { aux = b; b = c; c = aux; }
    /* Pode ter desarrumado a <= b; conserta */
    if (a > b) { aux = a; a = b; b = aux; }

    printf("Ordem crescente: %.2f %.2f %.2f\n", a, b, c);
    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
9 3 6	3,00 6,00 9,00
1 2 3	1,00 2,00 3,00 (já ordenado)
3 2 1	1,00 2,00 3,00 (ordem inversa)
5 5 1	1,00 5,00 5,00 (com repetição)

Exercício 5 — Quatro números em ordem decrescente

O que o enunciado pede

Receber três números obrigatoriamente em ordem e um quarto número que não segue essa regra. Mostrar os quatro em ordem decrescente.

Como pensar

1. O enunciado dá uma informação valiosa: os três primeiros já vêm ordenados. Isso significa que não é preciso ordenar tudo de novo — basta descobrir onde o quarto número se encaixa.
2. Assumindo que os três vêm em ordem crescente ($a \leq b \leq c$), existem quatro posições possíveis para o quarto valor d : depois de c , entre b e c , entre a e b , ou antes de a .
3. Cada uma dessas quatro possibilidades é um ramo de uma escada `else-if`, e cada ramo já imprime os quatro valores na ordem decrescente correta. Nenhuma troca de variáveis é necessária.
4. Repare como a informação do enunciado economizou trabalho: aproveitar o que já está garantido é uma habilidade que vale para a disciplina inteira.

Solução

```
/* Exercício 5 - Tres numeros ja ordenados + um quarto fora de ordem.
   Mostrar os quatro em ordem DECRESCENTE.
   Assume-se que os tres primeiros vem em ordem crescente (a <= b <= c).
   Basta descobrir onde o quarto valor (d) se encaixa. */
#include <stdio.h>

int main() {
    float a, b, c, d;

    printf("Digite tres numeros em ordem crescente: ");
    scanf("%f %f %f", &a, &b, &c);
    printf("Digite o quarto numero: ");
    scanf("%f", &d);

    printf("Ordem decrescente: ");
    if (d >= c)
        printf("%.2f %.2f %.2f %.2f\n", d, c, b, a);
    else if (d >= b)
        printf("%.2f %.2f %.2f %.2f\n", c, d, b, a);
    else if (d >= a)
        printf("%.2f %.2f %.2f %.2f\n", c, b, d, a);
    else
        printf("%.2f %.2f %.2f %.2f\n", c, b, a, d);

    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
2 4 6 e depois 5	6,00 5,00 4,00 2,00
2 4 6 e depois 9	9,00 6,00 4,00 2,00 (maior que todos)
2 4 6 e depois 1	6,00 4,00 2,00 1,00 (menor que todos)
2 4 6 e depois 3	6,00 4,00 3,00 2,00
2 4 6 e depois 6	6,00 6,00 4,00 2,00 (empate com o maior)

Exercício 6 — Par ou ímpar

O que o enunciado pede

Receber um número inteiro e verificar se ele é par ou ímpar.

Como pensar

1. O operador % devolve o resto da divisão inteira. Dividir por 2 só pode deixar resto 0 ou resto 1.
2. Resto 0 significa divisão exata, ou seja, número par. Qualquer outro resto significa ímpar.
3. Escreva `n % 2 == 0` e não `n % 2`, porque a comparação explícita é muito mais legível — e porque com números negativos alguns compiladores devolvem resto -1, que também é diferente de 0 e portanto ímpar, como esperado.
4. Generalizando: para saber se `x` é múltiplo de `n`, teste `x % n == 0`. Esse padrão reaparece em vários exercícios.

Solução

```
/* Exercício 6 - Par ou impar */
#include <stdio.h>

int main() {
    int n;

    printf("Digite um numero inteiro: ");
    scanf("%d", &n);

    if (n % 2 == 0)
        printf("%d e PAR\n", n);
    else
        printf("%d e IMPAR\n", n);

    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
4	4 é PAR
7	7 é ÍMPAR
0	0 é PAR
-3	-3 é ÍMPAR

Exercício 7 — Escrever A, B e C conforme o valor de I

O que o enunciado pede

Receber quatro valores: I, inteiro e positivo, e A, B e C, reais. Escrever os números A, B e C na forma indicada pela tabela. Supor que I seja sempre 1, 2 ou 3.

Forma de escrever conforme I

Valor de I	Forma de escrever
1	A, B e C em ordem crescente
2	A, B e C em ordem decrescente
3	O maior fica entre os outros dois números

Como pensar

1. A chave deste exercício é perceber que as três formas pedidas são reordenações dos mesmos três valores. Não é preciso escrever três algoritmos de ordenação diferentes.
2. Ordene uma única vez, com a técnica de trocas do exercício 4, até obter $a \leq b \leq c$. Depois disso, cada opção é apenas uma forma diferente de imprimir os mesmos três valores.
3. Com os valores ordenados: a opção 1 imprime a, b, c; a opção 2 imprime c, b, a; e a opção 3 imprime a, c, b — colocando o maior (c) no meio, exatamente como pede a tabela.
4. Separar “organizar os dados” de “apresentar os dados” é um bom hábito de projeto: o programa fica menor e muito mais fácil de conferir.

Solução

```
/* Exercício 7 - I define a forma de escrever A, B e C
   1 -> crescente | 2 -> decrescente | 3 -> o maior fica no meio */
#include <stdio.h>

int main() {
    int i;
    float a, b, c, aux;

    printf("Digite I (1, 2 ou 3): ");
    scanf("%d", &i);
    printf("Digite A, B e C: ");
    scanf("%f %f %f", &a, &b, &c);

    /* Ordena a <= b <= c uma unica vez; depois so escolhe a forma de exibir */
    if (a > b) { aux = a; a = b; b = aux; }
    if (b > c) { aux = b; b = c; c = aux; }
    if (a > b) { aux = a; a = b; b = aux; }

    if (i == 1)
        printf("%.2f %.2f %.2f\n", a, b, c);          /* crescente */
    else if (i == 2)
        printf("%.2f %.2f %.2f\n", c, b, a);          /* decrescente */
    else
        printf("%.2f %.2f %.2f\n", a, c, b);          /* maior no meio */

    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
l=1, valores 5 1 9	1,00 5,00 9,00
l=2, valores 5 1 9	9,00 5,00 1,00
l=3, valores 5 1 9	1,00 9,00 5,00
l=3, valores 2 2 8	2,00 8,00 2,00

Exercício 8 — Menu: somar dois números ou extrair raiz quadrada

O que o enunciado pede

Mostrar um menu com duas opções, receber a opção escolhida pelo usuário e os dados necessários, e executar a operação correspondente.

Como pensar

1. Este é o primeiro exercício de menu. A estrutura é sempre a mesma: imprimir as opções, ler a escolha, e desviar para o trecho certo.
2. Aqui a escolha é um valor exato de uma variável inteira, então tanto um switch quanto uma escada else-if resolvem. A solução usa if/else if para manter o exemplo simples; o exercício 13 mostra a versão com switch.
3. Não esqueça de tratar a opção inválida. Se o usuário digitar 7, o programa não pode simplesmente encerrar em silêncio — o último else existe para isso.
4. A raiz quadrada exige um cuidado extra: não existe raiz real de número negativo. Testar isso antes de chamar sqrt evita que o programa imprima a palavra “nan” na tela e deixe o aluno sem entender o que aconteceu.

Lembre-se do -lm

Este programa usa sqrt, então precisa de `#include <math.h>` no topo e, no Linux e no Mac, da opção `-lm` na compilação: `gcc ex08.c -o ex08 -lm`.

Solução

```
/* Exercício 8 - Menu: somar dois numeros / raiz quadrada */
#include <stdio.h>
#include <math.h>

int main() {
    int opcao;
    float x, y;

    printf("Menu de opcoes:\n");
    printf("1 - Somar dois numeros\n");
    printf("2 - Raiz quadrada de um numero\n");
    printf("Digite a opcao desejada -> ");
    scanf("%d", &opcao);

    if (opcao == 1) {
        printf("Digite os dois numeros: ");
        scanf("%f %f", &x, &y);
        printf("Soma = %.2f\n", x + y);
    } else if (opcao == 2) {
        printf("Digite o numero: ");
        scanf("%f", &x);
        if (x < 0)
            printf("Nao existe raiz quadrada real de numero negativo.\n");
        else
            printf("Raiz quadrada de %.2f = %.4f\n", x, sqrt(x));
    } else {
        printf("Opcao invalida!\n");
    }
    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
Opção 1, depois 7 e 5	Soma = 12,00
Opção 2, depois 16	Raiz quadrada de 16,00 = 4,0000
Opção 2, depois -9	Aviso de que não existe raiz real
Opção 7	Opção inválida!

Exercício 9 — Data e hora do sistema

O que o enunciado pede

Mostrar a data e a hora do sistema em dois formatos: dia/mês/ano numérico, e depois o mês por extenso, além da hora e do minuto.

Como pensar

1. Este exercício precisa de uma biblioteca nova: `time.h`. A função `time` devolve quantos segundos se passaram desde 1º de janeiro de 1970, e a função `localtime` converte esse número para uma estrutura com dia, mês, ano, hora e minuto separados.
2. Duas armadilhas da estrutura `tm`: o campo `tm_mon` vai de 0 a 11, então é preciso somar 1 para obter o mês de verdade; e o campo `tm_year` conta os anos a partir de 1900, então é preciso somar 1900.
3. A parte condicional do exercício é o mês por extenso: doze valores exatos de uma variável inteira. É o caso perfeito para um `switch` com doze `case`.
4. O formato `%02d` na função `printf` preenche com zero à esquerda quando o número tem um dígito só. É o que faz o programa escrever 05/03/2026 em vez de 5/3/2026.

Solução

```
/* Exercício 9 - Data e hora do sistema.
   Formato: dia/mes/ano - mes por extenso e hora:minuto */
#include <stdio.h>
#include <time.h>

int main() {
    time_t agora;
    struct tm *dh;
    int dia, mes, ano, hora, minuto;

    time(&agora);          /* segundos desde 01/01/1970 */
    dh = localtime(&agora); /* converte para data/hora local */

    dia    = dh->tm_mday;
    mes     = dh->tm_mon + 1; /* tm_mon vai de 0 a 11 */
    ano     = dh->tm_year + 1900; /* tm_year conta a partir de 1900 */
    hora    = dh->tm_hour;
    minuto  = dh->tm_min;

    printf("%02d/%02d/%d\n", dia, mes, ano);

    printf("%d de ", dia);
    switch (mes) {
        case 1: printf("janeiro"); break;
        case 2: printf("fevereiro"); break;
        case 3: printf("marco"); break;
        case 4: printf("abril"); break;
        case 5: printf("maio"); break;
        case 6: printf("junho"); break;
        case 7: printf("julho"); break;
        case 8: printf("agosto"); break;
        case 9: printf("setembro"); break;
        case 10: printf("outubro"); break;
        case 11: printf("novembro"); break;
        case 12: printf("dezembro"); break;
    }
    printf(" de %d\n", ano);

    printf("%02d:%02d\n", hora, minuto);
    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
(sem entrada)	Data numérica, por exemplo 31/08/2026
(sem entrada)	Data por extenso, por exemplo 31 de agosto de 2026
(sem entrada)	Hora no formato hh:mm, por exemplo 19:15

Exercício 10 — A data cronologicamente maior

O que o enunciado pede

Determinar a data cronologicamente maior entre duas datas fornecidas pelo usuário. Cada data é composta por três valores inteiros: dia, mês e ano.

Como pensar

1. O erro clássico aqui é comparar os campos isoladamente. A data 31/12/2019 tem o maior dia e o maior mês, mas é menor que 01/01/2020.
2. A comparação de datas obedece a uma hierarquia: o ano decide primeiro; se os anos empatarem, o mês decide; se o mês também empatar, aí sim o dia decide.
3. Traduzindo em uma expressão: a primeira data é maior quando o ano dela é maior, OU quando os anos são iguais e o mês dela é maior, OU quando ano e mês são iguais e o dia dela é maior.
4. Os parênteses são essenciais nessa expressão. Sem eles, a precedência do && sobre o || produz um agrupamento diferente do pretendido e o programa erra em alguns casos.

Um padrão que reaparece

Comparar do campo mais significativo para o menos significativo é o mesmo raciocínio usado para comparar horários, números de versão de software e placares de campeonato. Vale guardar o padrão, não só a solução.

Solução

```
/* Exercício 10 - Data cronologicamente maior */
#include <stdio.h>

int main() {
    int d1, m1, a1, d2, m2, a2;

    printf("Digite a primeira data (dia mes ano): ");
    scanf("%d %d %d", &d1, &m1, &a1);
    printf("Digite a segunda data (dia mes ano): ");
    scanf("%d %d %d", &d2, &m2, &a2);

    /* Compara do campo mais significativo para o menos significativo */
    if (a1 > a2 || (a1 == a2 && (m1 > m2 || (m1 == m2 && d1 > d2))))
        printf("A data maior e %02d/%02d/%d\n", d1, m1, a1);
    else if (a1 == a2 && m1 == m2 && d1 == d2)
        printf("As datas sao iguais: %02d/%02d/%d\n", d1, m1, a1);
    else
        printf("A data maior e %02d/%02d/%d\n", d2, m2, a2);

    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
01 01 2020 e 31 12 2019	A data maior é 01/01/2020
05 03 2020 e 06 03 2020	A data maior é 06/03/2020
05 03 2020 e 05 03 2020	As datas são iguais

Entrada	Saída esperada
10 05 2021 e 10 04 2021	A data maior é 10/05/2021 (mesmo ano, mês decide)

Exercício 11 — Duração de um jogo

O que o enunciado pede

Receber a hora de início e a hora final de um jogo, cada uma composta por duas variáveis inteiras (hora e minuto), e calcular a duração. O jogo dura no máximo 24 horas e pode começar em um dia e terminar no dia seguinte.

Como pensar

1. Subtrair horas de horas e minutos de minutos separadamente gera um problema chato: quando os minutos finais são menores que os iniciais, é preciso “pegar emprestado” 60 minutos da hora, exatamente como na subtração armada da escola.
2. Existe um truque que elimina esse problema inteiro: converta tudo para minutos. Uma hora e trinta vira 90. Feita a conta em minutos, converta o resultado de volta dividindo por 60 (horas) e tomando o resto da divisão por 60 (minutos).
3. Resta tratar a virada do dia. Se o horário final for maior que o inicial, o jogo terminou no mesmo dia e basta subtrair. Se for menor, o jogo atravessou a meia-noite: some o que faltava para o fim do dia com o que se passou no dia seguinte.
4. E se os dois horários forem iguais? Pelo enunciado, o jogo dura no máximo 24 horas, então a interpretação correta é que ele durou exatamente 24 horas — e não zero.

Solução

```
/* Exercício 11 - Duracao de um jogo (pode virar o dia) */
#include <stdio.h>

int main() {
    int hi, mi, hf, mf;
    int inicio, fim, duracao;

    printf("Hora de inicio (hora minuto): ");
    scanf("%d %d", &hi, &mi);
    printf("Hora final (hora minuto): ");
    scanf("%d %d", &hf, &mf);

    /* Converte tudo para minutos: elimina a necessidade de tratar
       emprestimo entre horas e minutos separadamente. */
    inicio = hi * 60 + mi;
    fim    = hf * 60 + mf;

    if (fim > inicio)
        duracao = fim - inicio;           /* mesmo dia */
    else if (fim < inicio)
        duracao = (24 * 60) - inicio + fim; /* virou o dia */
    else
        duracao = 24 * 60;                 /* durou exatamente 24 h */

    printf("Duracao do jogo: %d hora(s) e %d minuto(s)\n",
           duracao / 60, duracao % 60);
    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
10:00 até 12:30	2 hora(s) e 30 minuto(s)
22:30 até 01:15	2 hora(s) e 45 minuto(s) — virou o dia

Entrada	Saída esperada
23:50 até 00:10	0 hora(s) e 20 minuto(s)
10:00 até 10:00	24 hora(s) e 0 minuto(s)
10:45 até 12:30	1 hora(s) e 45 minuto(s) — sem empréstimo

Exercício 12 — Aumento salarial por cargo

O que o enunciado pede

Receber o código do cargo de um funcionário e seu salário atual, e mostrar o cargo por extenso, o valor do aumento e o novo salário.

Cargos e percentuais de aumento

Código	Cargo	Percentual
1	Escriturário	50%
2	Secretário	35%
3	Caixa	20%
4	Gerente	10%
5	Diretor	Não tem aumento

Como pensar

1. Os códigos são valores exatos de uma variável inteira, então o switch é a construção natural.
2. Como ainda não estudamos vetores nem cadeias de caracteres, o nome do cargo é impresso diretamente dentro de cada case, em vez de ser guardado em uma variável de texto.
3. O diretor é um caso interessante: ele não tem aumento, mas isso não é o mesmo que “código inválido”. O percentual dele é zero, e o programa deve mostrar normalmente o cargo, um aumento de R\$ 0,00 e o novo salário igual ao antigo.
4. Para distinguir “sem aumento” de “código inválido”, a solução inicializa o percentual com -1 e usa esse valor como marcador. Se depois do switch o percentual continuar negativo, é porque nenhum case foi executado.
5. Atenção ao converter percentual em fator: 50% é 0,50 e não 50. Multiplicar o salário por 50 renderia um aumento de cinco mil por cento.

Solução

```
/* Exercício 12 - Aumento por cargo
   Obs.: ainda não usamos vetores/strings, então o nome do cargo
   é impresso diretamente dentro de cada case. */
#include <stdio.h>

int main() {
    int codigo;
    float salario, percentual;

    printf("Codigo do cargo (1 a 5): ");
    scanf("%d", &codigo);
    printf("Salario atual: ");
    scanf("%f", &salario);

    percentual = -1.0; /* -1 marca "codigo invalido" */

    switch (codigo) {
        case 1: printf("Cargo: Escriturario\n"); percentual = 0.50; break;
        case 2: printf("Cargo: Secretario\n");   percentual = 0.35; break;
        case 3: printf("Cargo: Caixa\n");        percentual = 0.20; break;
        case 4: printf("Cargo: Gerente\n");       percentual = 0.10; break;
        case 5: printf("Cargo: Diretor\n");       percentual = 0.00; break;
        default: printf("Codigo de cargo invalido!\n");
    }
}
```

```
if (percentual >= 0.0) {  
    printf("Valor do aumento: R$ %.2f\n", salario * percentual);  
    printf("Novo salario: R$ %.2f\n", salario + salario * percentual);  
}  
return 0;  
}
```

Teste você mesmo

Entrada	Saída esperada
Código 1, salário 1000	Escriturário — aumento R\$ 500,00 — novo R\$ 1500,00
Código 4, salário 1000	Gerente — aumento R\$ 100,00 — novo R\$ 1100,00
Código 5, salário 1000	Diretor — aumento R\$ 0,00 — novo R\$ 1000,00
Código 9, salário 1000	Código de cargo inválido!

Exercício 13 — Menu: imposto, novo salário ou classificação

O que o enunciado pede

Apresentar um menu de três opções, permitir que o usuário escolha, receber o salário e executar a operação escolhida. Tratar a possibilidade de opção inválida.

Opção 1 — percentual do imposto

Salário	Percentual do imposto
Menor que R\$ 500,00	5%
De R\$ 500,00 a R\$ 850,00	10%
Acima de R\$ 850,00	15%

Opção 2 — valor do aumento

Salário	Aumento
Maiores que R\$ 1.500,00	R\$ 25,00
De R\$ 750,00 (inclusive) a R\$ 1.500,00	R\$ 50,00
De R\$ 450,00 (inclusive) a R\$ 750,00	R\$ 75,00
Menores que R\$ 450,00	R\$ 100,00

Opção 3 — classificação

Salário	Classificação
Até R\$ 700,00 (inclusive)	Mal remunerado
Maiores que R\$ 700,00	Bem remunerado

Como pensar

1. Este exercício combina as duas construções da aula: um switch para escolher a opção do menu, e uma escada else-if dentro de cada opção para tratar as faixas de salário.
2. A leitura do salário é comum às três opções, então ela pode ser feita uma única vez antes do switch — desde que só aconteça quando a opção for válida. É por isso que ela aparece dentro de um if que verifica se a opção está entre 1 e 3.
3. Preste muita atenção às palavras “inclusive” das tabelas. Na opção 2, R\$ 750,00 pertence à faixa de R\$ 50,00 e não à de R\$ 75,00; por isso o teste é `salario >= 750.0`. Na opção 1, R\$ 500,00 pertence à faixa de 10%; por isso o primeiro teste é `salario < 500.0`, com menor estrito.
4. Na opção 2 as faixas são testadas da mais alta para a mais baixa; na opção 1, da mais baixa para a mais alta. As duas ordens funcionam, desde que sejam coerentes com o sinal usado na comparação.

A fronteira é onde mora o erro

Nas três tabelas deste exercício, os valores de fronteira são 500, 850, 450, 750, 1500 e 700. Teste cada um deles, além de um valor imediatamente acima e outro imediatamente abaixo. São cerca de vinte testes e eles pegam praticamente qualquer erro possível neste exercício.

Solução

```
/* Exercício 13 - Menu: imposto / novo salario / classificacao */
#include <stdio.h>

int main() {
    int opcao;
    float salario, imposto, aumento;

    printf("Menu de opcoes:\n");
    printf("1 - Imposto\n");
    printf("2 - Novo Salario\n");
    printf("3 - Classificacao\n");
    printf("Digite a opcao desejada -> ");
    scanf("%d", &opcao);

    if (opcao >= 1 && opcao <= 3) {
        printf("Digite o salario: ");
        scanf("%f", &salario);
    }

    switch (opcao) {
        case 1:
            if (salario < 500.0)
                imposto = salario * 0.05;
            else if (salario <= 850.0)
                imposto = salario * 0.10;
            else
                imposto = salario * 0.15;
            printf("Valor do imposto: R$ %.2f\n", imposto);
            break;

        case 2:
            if (salario > 1500.0)
                aumento = 25.0;
            else if (salario >= 750.0)
                aumento = 50.0;
            else if (salario >= 450.0)
                aumento = 75.0;
            else
                aumento = 100.0;
            printf("Aumento: R$ %.2f\n", aumento);
            printf("Novo salario: R$ %.2f\n", salario + aumento);
            break;

        case 3:
            if (salario <= 700.0)
                printf("Mal remunerado\n");
            else
                printf("Bem remunerado\n");
            break;

        default:
            printf("Opcao invalida!\n");
    }
    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
Opção 1, salário 499	Imposto de 5% = R\$ 24,95
Opção 1, salário 500	Imposto de 10% = R\$ 50,00 (fronteira)
Opção 1, salário 850	Imposto de 10% = R\$ 85,00 (fronteira)
Opção 1, salário 851	Imposto de 15% = R\$ 127,65

Entrada	Saída esperada
Opção 2, salário 750	Aumento R\$ 50,00 (fronteira inclusive)
Opção 2, salário 1500	Aumento R\$ 50,00 (fronteira)
Opção 2, salário 1501	Aumento R\$ 25,00
Opção 2, salário 449	Aumento R\$ 100,00
Opção 3, salário 700	Mal remunerado (fronteira inclusive)
Opção 3, salário 701	Bem remunerado
Opção 9	Opção inválida!

Exercício 14 — Novo salário com bonificação e auxílio-escola

O que o enunciado pede

Receber o salário de um funcionário e mostrar o novo salário, acrescido de bonificação e de auxílio-escola.

Bonificação

Salário	Bonificação
Até R\$ 500,00	5% do salário
Entre R\$ 500,01 e R\$ 1.200,00	12% do salário
Acima de R\$ 1.200,00	Sem bonificação

Auxílio-escola

Salário	Auxílio-escola
Até R\$ 600,00	R\$ 150,00
Mais que R\$ 600,00	R\$ 100,00

Como pensar

1. O ponto central deste exercício é perceber que as duas tabelas são independentes uma da outra. Elas usam o mesmo salário como referência, mas têm fronteiras diferentes: 500 e 1200 na primeira, apenas 600 na segunda.
2. Portanto são duas escadas else-if separadas, uma após a outra — e não uma escada aninhada dentro da outra. Tentar combiná-las produz um emaranhado de casos e quase sempre leva a erro.
3. Um salário de R\$ 550,00, por exemplo, recebe bonificação de 12% (segunda faixa da primeira tabela) e auxílio de R\$ 150,00 (primeira faixa da segunda tabela). As faixas não se alinham, e está tudo certo.
4. O novo salário é a soma dos três valores: salário original, bonificação e auxílio.

Solução

```
/* Exercício 14 - Bonificacao + auxilio-escola */
#include <stdio.h>

int main() {
    float salario, bonificacao, auxilio, novo;

    printf("Digite o salario: ");
    scanf("%f", &salario);

    /* Bonificacao */
    if (salario <= 500.0)
        bonificacao = salario * 0.05;
    else if (salario <= 1200.0)
        bonificacao = salario * 0.12;
    else
        bonificacao = 0.0;

    /* Auxilio-escola (tabela independente da anterior) */
    if (salario <= 600.0)
        auxilio = 150.0;
    else
        auxilio = 100.0;
```

```

    novo = salario + bonificacao + auxilio;

    printf("Bonificacao: R$ %.2f\n", bonificacao);
    printf("Auxilio-escola: R$ %.2f\n", auxilio);
    printf("Novo salario: R$ %.2f\n", novo);
    return 0;
}

```

Teste você mesmo

Entrada	Saída esperada
500	Bonificação R\$ 25,00 (5%) + auxílio R\$ 150,00
501	Bonificação R\$ 60,12 (12%) + auxílio R\$ 150,00
600	Bonificação R\$ 72,00 (12%) + auxílio R\$ 150,00 (fronteira)
601	Bonificação R\$ 72,12 (12%) + auxílio R\$ 100,00
1200	Bonificação R\$ 144,00 (12%) + auxílio R\$ 100,00 (fronteira)
1201	Sem bonificação + auxílio R\$ 100,00

Exercício 15 — Folha de pagamento completa

O que o enunciado pede

Receber o valor do salário mínimo, o número de horas trabalhadas, o número de dependentes e a quantidade de horas extras. Calcular e mostrar o salário a receber, seguindo as regras abaixo.

Imposto de renda retido na fonte (sobre o salário bruto)

Salário bruto	IRRF
Inferior a R\$ 200,00	Isento
De R\$ 200,00 até R\$ 500,00	10%
Superior a R\$ 500,00	20%

Gratificação (sobre o salário líquido)

Salário líquido	Gratificação
Até R\$ 350,00	R\$ 100,00
Superior a R\$ 350,00	R\$ 50,00

Como pensar

1. Este é o exercício mais longo da lista, mas não o mais difícil. Ele é longo porque tem muitas etapas — e a estratégia certa é justamente respeitar essa sequência, calculando um valor de cada vez e guardando cada um em sua própria variável.
2. A ordem das contas é: valor da hora (um quinto do salário mínimo); salário do mês (horas vezes valor da hora); valor dos dependentes (R\$ 32,00 por dependente); valor das horas extras (cada uma vale a hora normal acrescida de 50%, ou seja, o valor da hora multiplicado por 1,5).
3. O salário bruto é a soma do salário do mês com o valor dos dependentes e o das horas extras. Só depois de ter o bruto é possível calcular o IRRF.
4. Atenção a uma sutileza: o IRRF incide sobre o salário bruto, mas a gratificação depende do salário líquido — que só existe depois de descontar o IRRF. As duas tabelas se aplicam a bases diferentes, em momentos diferentes. Trocar a ordem faz o programa errar.
5. Resista à tentação de escrever tudo em uma expressão gigante. Uma variável por etapa deixa o programa mais longo, mas infinitamente mais fácil de conferir e corrigir.

Conferindo passo a passo

Com salário mínimo R\$ 1.000,00, 100 horas, 2 dependentes e 10 horas extras: valor da hora R\$ 200,00; salário do mês R\$ 20.000,00; dependentes R\$ 64,00; horas extras $10 \times 300 = \text{R\$ } 3.000,00$; bruto R\$ 23.064,00; IRRF de 20% = R\$ 4.612,80; líquido R\$ 18.451,20; gratificação R\$ 50,00; total R\$ 18.501,20.

Solução

```
/* Exercício 15 - Folha de pagamento completa */
#include <stdio.h>

int main() {
    float salario_minimo, valor_hora, salario_mes;
```

```

float valor_dependentes, valor_extras, salario_bruto;
float irrf, salario_liquido, gratificacao, total;
int horas, dependentes, horas_extras;

printf("Salario minimo: ");
scanf("%f", &salario_minimo);
printf("Numero de horas trabalhadas: ");
scanf("%d", &horas);
printf("Numero de dependentes: ");
scanf("%d", &dependentes);
printf("Quantidade de horas extras: ");
scanf("%d", &horas_extras);

valor_hora      = salario_minimo / 5.0;
salario_mes     = horas * valor_hora;
valor_dependentes = dependentes * 32.0;
valor_extras    = horas_extras * (valor_hora * 1.5);
salario_bruto    = salario_mes + valor_dependentes + valor_extras;

/* IRRF sobre o salario bruto */
if (salario_bruto < 200.0)
    irrf = 0.0;                                /* isento */
else if (salario_bruto <= 500.0)
    irrf = salario_bruto * 0.10;
else
    irrf = salario_bruto * 0.20;

salario_liquido = salario_bruto - irrf;

/* Gratificacao sobre o salario liquido */
if (salario_liquido <= 350.0)
    gratificacao = 100.0;
else
    gratificacao = 50.0;

total = salario_liquido + gratificacao;

printf("\n--- Demonstrativo ---\n");
printf("Valor da hora trabalhada: R$ %.2f\n", valor_hora);
printf("Salario do mes.....: R$ %.2f\n", salario_mes);
printf("Dependentes.....: R$ %.2f\n", valor_dependentes);
printf("Horas extras.....: R$ %.2f\n", valor_extras);
printf("Salario bruto.....: R$ %.2f\n", salario_bruto);
printf("IRRF.....: R$ %.2f\n", irrf);
printf("Salario liquido.....: R$ %.2f\n", salario_liquido);
printf("Gratificacao.....: R$ %.2f\n", gratificacao);
printf("Total a receber.....: R$ %.2f\n", total);
return 0;
}

```

Teste você mesmo

Entrada	Saída esperada
1000 / 100h / 2 dep / 10 he	Total a receber R\$ 18.501,20
1000 / 1h / 0 dep / 0 he	Bruto R\$ 200,00 — IRRF 10% (fronteira)
500 / 1h / 0 dep / 0 he	Bruto R\$ 100,00 — isento de IRRF
1000 / 2h / 0 dep / 0 he	Bruto R\$ 400,00 — IRRF 10% — líquido R\$ 360,00 — gratificação R\$ 50,00

Exercício 16 — Reajuste de preços no supermercado

O que o enunciado pede

Receber o preço atual e a venda mensal média de um produto e calcular o novo preço. Para ter o preço alterado, o produto precisa preencher pelo menos um dos requisitos da tabela.

Requisitos e reajustes

Venda média mensal	Preço atual	% de aumento	% de diminuição
menor que 500	menor que R\$ 30,00	10	—
de 500 a menos de 1.200	de R\$ 30,00 a menos de R\$ 80,00	15	—
1.200 ou mais	R\$ 80,00 ou mais	—	20

Como pensar

1. A expressão-chave do enunciado é “pelo menos um dos requisitos”. Isso significa OU, e não E — portanto o operador é `||`, não `&&`.
2. Um produto que vende 1.500 unidades por mês entra na terceira faixa mesmo que custe R\$ 10,00, porque já preencheu um dos dois requisitos.
3. Como um produto pode satisfazer requisitos de faixas diferentes ao mesmo tempo, a ordem dos testes passa a importar de verdade. A solução testa da faixa mais forte (a redução de 20%) para a mais fraca, de modo que o caso mais extremo tenha prioridade.
4. Para aplicar um aumento de 15%, multiplique por 1,15. Para uma redução de 20%, multiplique por 0,80. Escrever `preco = preco + preco * 0.15` dá o mesmo resultado, mas a forma com um único fator é mais curta e menos sujeita a erro de digitação.

Solução

```
/* Exercício 16 - Reajuste de preço no supermercado.
   Basta preencher PELO MENOS UM requisito -> operador || .
   As faixas são testadas da mais forte (20% de redução) para a mais fraca. */
#include <stdio.h>

int main() {
    float preco, venda, novo;

    printf("Preço atual do produto: ");
    scanf("%f", &preco);
    printf("Venda mensal média: ");
    scanf("%f", &venda);

    if (venda >= 1200.0 || preco >= 80.0) {
        novo = preco * 0.80; /* diminui 20% */
        printf("Reajuste: -20%%\n");
    } else if ((venda >= 500.0 && venda < 1200.0) ||
               (preco >= 30.0 && preco < 80.0)) {
        novo = preco * 1.15; /* aumenta 15% */
        printf("Reajuste: +15%%\n");
    } else {
        novo = preco * 1.10; /* aumenta 10% */
        printf("Reajuste: +10%%\n");
    }

    printf("Preço antigo: R$ %.2f\n", preco);
    printf("Novo preço: R$ %.2f\n", novo);
    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
Preço 10, venda 100	Aumento de 10% — novo preço R\$ 11,00
Preço 10, venda 600	Aumento de 15% — pela venda, apesar do preço baixo
Preço 30, venda 100	Aumento de 15% — pelo preço (fronteira)
Preço 80, venda 100	Redução de 20% — pelo preço (fronteira)
Preço 10, venda 1200	Redução de 20% — pela venda (fronteira)
Preço 90, venda 100	Redução de 20% — novo preço R\$ 72,00

Exercício 17 — Equação do segundo grau

O que o enunciado pede

Resolver equações do segundo grau na forma $a \cdot x^2 + b \cdot x + c = 0$. O coeficiente a deve ser diferente de zero. Se o discriminante for negativo não existe raiz real; se for zero existe uma raiz; se for positivo existem duas.

Como pensar

1. Antes de qualquer conta, verifique se a é diferente de zero. Além de ser exigência do enunciado, dividir por $2 \cdot a$ com a igual a zero causaria uma divisão por zero.
2. Calcule o discriminante $\Delta = b^2 - 4 \cdot a \cdot c$ e guarde-o em uma variável. Ele será consultado três vezes; recalculá-lo a cada teste é desperdício e fonte de erro de digitação.
3. Os três casos formam uma escada else-if de três ramos: Δ menor que zero, Δ igual a zero, e o resto (Δ maior que zero).
4. Para Δ igual a zero, a fórmula se reduz a $x = -b / (2 \cdot a)$, porque a raiz de zero é zero. Não é preciso chamar sqrt nesse ramo.
5. Cuidado com os parênteses na fórmula das raízes. Escrever $-b + \text{sqrt}(\text{delta}) / (2 \cdot a)$ está errado: pela precedência, a divisão aconteceria antes da soma. O numerador inteiro precisa dos seus próprios parênteses.

Sobre comparar Δ com zero

O teste $\text{delta} == 0$ usa igualdade entre números reais, algo que este material recomenda evitar. Aqui ele é aceitável porque é o que o enunciado pede e porque, com entradas inteiras digitadas pelo usuário, o cálculo costuma dar exatamente zero. Em um programa de produção, o teste correto seria $\text{fabs}(\text{delta}) < 0.0001$.

Solução

```
/* Exercício 17 - Equacao do 2o grau */
#include <stdio.h>
#include <math.h>

int main() {
    float a, b, c, delta, x1, x2, x;

    printf("Digite os coeficientes a, b e c: ");
    scanf("%f %f %f", &a, &b, &c);

    if (a == 0) {
        printf("O coeficiente 'a' deve ser diferente de zero.\n");
    } else {
        delta = b * b - 4 * a * c;
        printf("Delta = %.2f\n", delta);

        if (delta < 0) {
            printf("Nao existe raiz real.\n");
        } else if (delta == 0) {
            x = -b / (2 * a);
            printf("Existe uma raiz real: x = %.4f\n", x);
        } else {
            x1 = (-b + sqrt(delta)) / (2 * a);
            x2 = (-b - sqrt(delta)) / (2 * a);
            printf("Existem duas raizes reais:\n");
        }
    }
}
```

```
        printf("x1 = %.4f\n", x1);  
        printf("x2 = %.4f\n", x2);  
    }  
    return 0;  
}
```

Teste você mesmo

Entrada	Saída esperada
1 -3 2	Duas raízes: x1 = 2,0000 e x2 = 1,0000
1 2 1	Uma raiz: x = -1,0000
1 0 5	Não existe raiz real
0 2 1	Aviso de que a deve ser diferente de zero

Exercício 18 — Classificação de triângulos

O que o enunciado pede

Dados três valores X, Y e Z, verificar se eles podem ser os comprimentos dos lados de um triângulo e, em caso afirmativo, classificá-lo como equilátero, isósceles ou escaleno. Se não formarem um triângulo, escrever uma mensagem.

Como pensar

1. São duas decisões em sequência, e a ordem entre elas é obrigatória: primeiro verificar se o triângulo existe, e só depois classificá-lo. Não faz sentido perguntar se um triângulo é isósceles antes de saber se ele é um triângulo.
2. A condição de existência exige que cada lado seja menor que a soma dos outros dois. São três comparações, todas obrigatórias ao mesmo tempo — logo, ligadas por &&.
3. Dentro do ramo de existência vem a classificação, e aqui a ordem também importa. Teste equilátero primeiro: um triângulo com três lados iguais também tem dois lados iguais, e se o teste de isósceles vier antes, nenhum triângulo será classificado como equilátero.
4. O último caso, escaleno, não precisa de condição: se não é equilátero nem isósceles, só resta ser escaleno. É o else final da escada.

Solução

```
/* Exercício 18 - Classificacao de triangulos */
#include <stdio.h>

int main() {
    float x, y, z;

    printf("Digite os tres lados (X Y Z): ");
    scanf("%f %f %f", &x, &y, &z);

    /* Condicao de existencia: cada lado menor que a soma dos outros dois */
    if (x < y + z && y < x + z && z < x + y) {
        if (x == y && y == z)
            printf("Triangulo EQUILATERO\n");
        else if (x == y || y == z || x == z)
            printf("Triangulo ISOSCELES\n");
        else
            printf("Triangulo ESCALENO\n");
    } else {
        printf("Estes valores nao formam um triangulo.\n");
    }
    return 0;
}
```

Teste você mesmo

Entrada	Saída esperada
3 3 3	Triângulo EQUILÁTERO
3 3 5	Triângulo ISÓSCELES
3 4 5	Triângulo ESCALENO
1 2 10	Estes valores não formam um triângulo
1 2 3	Não formam triângulo (caso degenerado: $3 = 1 + 2$)

Exercício 19 — Classificação por altura e peso

O que o enunciado pede

Receber a altura e o peso de uma pessoa e mostrar sua classificação de acordo com a tabela de dupla entrada abaixo.

Classificação (linhas: altura / colunas: peso)

Altura	Peso até 60	Peso entre 60 e 90 (inclusive)	Peso acima de 90
Menores que 1,20	A	D	G
De 1,20 a 1,70	B	E	H
Maiores que 1,70	C	F	I

Como pensar

1. Uma tabela de dupla entrada vira um if aninhado em dois níveis. O de fora escolhe a linha (a altura) e o de dentro escolhe a coluna (o peso) dentro daquela linha.
2. São três faixas de altura e três de peso, o que dá nove combinações — e nove é exatamente o número de letras da tabela. Se o seu programa não tiver nove caminhos possíveis, algum caso ficou de fora.
3. Escrever nove condições combinadas com && também funciona, mas fica muito mais longo e mais fácil de errar. O aninhamento aproveita o fato de que, escolhida a linha, só restam três possibilidades.
4. Este exercício tem uma armadilha real de ponto flutuante, explicada no quadro abaixo. Leia com atenção antes de codificar.

Por que este programa usa double, e não float

Os limites da tabela — 1,20 e 1,70 — são números quebrados. Um float guarda 1,70 como 1,70000004..., enquanto a constante 1.70 escrita no código é um double, guardado como 1,69999999.... O primeiro é maior que o segundo, então o teste `altura <= 1.70` resulta FALSO justamente quando o usuário digita exatamente 1,70 — e a pessoa é classificada na faixa errada. Usando double e lendo com `%lf`, os dois lados passam a ser o mesmo número e o teste funciona. Este erro é real: ele aparece se você escrever o programa com float.

Solução

```
/* Exercício 19 - Classificacao por altura x peso (tabela de dupla entrada)
ATENCAO: usamos DOUBLE (e nao float) porque os limites da tabela
(1,20 e 1,70) sao numeros quebrados. Com float, o valor 1.70 digitado
e guardado como 1.70000004..., e o teste (altura <= 1.70) daria FALSO
justamente na fronteira. Com double o problema desaparece.
Repare que a leitura de double usa %lf, e nao %f. */
#include <stdio.h>

int main() {
    double altura, peso;
    char classificacao;

    printf("Digite a altura (m): ");
    scanf("%lf", &altura);
    printf("Digite o peso (kg): ");
```

```

scanf("%lf", &peso);

/* Estrutura: primeiro escolhe a LINHA (altura),
   depois a COLUNA (peso) dentro dela. */
if (altura < 1.20) {
    if (peso <= 60.0)      classificacao = 'A';
    else if (peso <= 90.0) classificacao = 'D';
    else                  classificacao = 'G';
} else if (altura <= 1.70) {
    if (peso <= 60.0)      classificacao = 'B';
    else if (peso <= 90.0) classificacao = 'E';
    else                  classificacao = 'H';
} else {
    if (peso <= 60.0)      classificacao = 'C';
    else if (peso <= 90.0) classificacao = 'F';
    else                  classificacao = 'I';
}

printf("Classificacao: %c\n", classificacao);
return 0;
}

```

Teste você mesmo

Entrada	Saída esperada
1,10 e 50	A
1,10 e 75	D
1,10 e 95	G
1,20 e 50	B (fronteira de altura)
1,70 e 90	E (dupla fronteira)
1,70 e 91	H (fronteira crítica — falha com float)
1,71 e 50	C
1,80 e 90	F
1,80 e 91	I

Exercício 20 — Produto importado: peso, preço e imposto

O que o enunciado pede

Receber o código de um produto (inteiro entre 1 e 10), o peso em quilos e o código do país de origem (entre 1 e 3). Calcular e mostrar o peso convertido em gramas, o preço total, o valor do imposto — cobrado sobre o preço total e dependente do país — e o valor total.

Imposto por país de origem

Código do país	Imposto
1	0%
2	15%
3	25%

Preço por grama, conforme o código do produto

Código do produto	Preço por grama
1 a 4	10
5 a 7	25
8 a 10	35

Como pensar

1. Repare que as duas tabelas são de naturezas diferentes. A do país lista valores exatos (1, 2 ou 3) e pede switch ou uma comparação por igualdade. A do produto lista faixas (1 a 4, 5 a 7, 8 a 10) e pede uma escada else-if.
2. Como o enunciado garante que o código do produto está sempre entre 1 e 10, os testes podem ser simplificados: basta perguntar se é menor ou igual a 4, depois se é menor ou igual a 7, e o resto cai no else.
3. O preço é por grama, mas o peso é informado em quilos. Converta primeiro (um quilo são mil gramas) e só então multiplique pelo preço unitário. Esquecer essa conversão faz o resultado sair mil vezes menor.
4. Siga a ordem das perguntas do enunciado ao imprimir: peso em gramas, preço total, valor do imposto e valor total. Programas que respondem exatamente o que foi pedido, na ordem pedida, são mais fáceis de corrigir.

Solução

```
/* Exercício 20 - Produto importado: peso, preço, imposto e total */
#include <stdio.h>

int main() {
    int cod_produto, cod_pais;
    float peso_kg, peso_g, preco_grama, preco_total;
    float aliquota, imposto, total;

    printf("Codigo do produto (1 a 10): ");
    scanf("%d", &cod_produto);
    printf("Peso do produto (kg): ");
    scanf("%f", &peso_kg);
    printf("Codigo do pais de origem (1 a 3): ");
    scanf("%d", &cod_pais);
```

```

    peso_g = peso_kg * 1000.0;

    /* Preço por grama depende da faixa do código do produto */
    if (cod_produto <= 4)
        preco_grama = 10.0;
    else if (cod_produto <= 7)
        preco_grama = 25.0;
    else
        preco_grama = 35.0;

    preco_total = peso_g * preco_grama;

    /* Imposto depende do país */
    if (cod_pais == 1)
        aliquota = 0.00;
    else if (cod_pais == 2)
        aliquota = 0.15;
    else
        aliquota = 0.25;

    imposto = preco_total * aliquota;
    total    = preco_total + imposto;

    printf("\nPeso em gramas...: %.2f g\n", peso_g);
    printf("Preço total.....: R$ %.2f\n", preco_total);
    printf("Valor do imposto.: R$ %.2f\n", imposto);
    printf("Valor total.....: R$ %.2f\n", total);
    return 0;
}

```

Teste você mesmo

Entrada	Saída esperada
Produto 5, 2 kg, país 2	2000 g — R\$ 50.000,00 — imposto R\$ 7.500,00 — total R\$ 57.500,00
Produto 1, 1 kg, país 1	1000 g — R\$ 10.000,00 — imposto R\$ 0,00 — total R\$ 10.000,00
Produto 4, 1 kg, país 3	Preço por grama 10 (fronteira da primeira faixa)
Produto 8, 1 kg, país 3	Preço por grama 35 — imposto de 25%

Exercício 21 — Carga de caminhão: peso, preço e imposto

O que o enunciado pede

Receber o código do estado de origem (inteiro entre 1 e 5), o peso da carga em toneladas e o código da carga (inteiro entre 10 e 40). Calcular e mostrar o peso convertido em quilos, o preço da carga, o valor do imposto — cobrado sobre o preço e dependente do estado — e o valor total transportado.

Imposto por estado de origem

Código do estado	Imposto
1	35%
2	25%
3	15%
4	5%
5	Isento

Preço por quilo, conforme o código da carga

Código da carga	Preço por quilo
10 a 20	100
21 a 30	250
31 a 40	340

Como pensar

1. Este exercício é gêmeo do anterior. Se você resolveu o 20, já sabe resolver este: muda a unidade (toneladas em vez de quilos), mudam os números das tabelas, mas a estrutura é idêntica.
2. Reconhecer que dois problemas têm a mesma estrutura é uma das habilidades mais valiosas da programação. Vale mais do que decorar soluções individuais.
3. Uma tonelada são mil quilos, então a conversão é a mesma multiplicação por 1000 do exercício anterior.
4. O código do estado tem cinco valores exatos, o que torna o switch a escolha mais limpa. A carga é dividida em três faixas, o que pede else-if. O programa usa cada construção onde ela se encaixa melhor.
5. O estado 5 é isento, ou seja, alíquota zero — e não “sem imposto a calcular”. O valor do imposto deve ser exibido normalmente como R\$ 0,00.

Solução

```
/* Exercício 21 - Carga de caminhão: peso, preço, imposto e total
   Mesma estrutura do exercício 20 (faixas + tabela por código). */
#include <stdio.h>

int main() {
    int cod_estado, cod_carga;
    float peso_ton, peso_kg, preco_quilo, preco_carga;
    float aliquota, imposto, total;

    printf("Codigo do estado de origem (1 a 5): ");
    scanf("%d", &cod_estado);
    printf("Peso da carga (toneladas): ");
```

```

scanf("%f", &peso_ton);
printf("Codigo da carga (10 a 40): ");
scanf("%d", &cod_carga);

peso_kg = peso_ton * 1000.0;

/* Preco por quilo depende da faixa do codigo da carga */
if (cod_carga <= 20)
    preco_quilo = 100.0;
else if (cod_carga <= 30)
    preco_quilo = 250.0;
else
    preco_quilo = 340.0;

preco_carga = peso_kg * preco_quilo;

/* Imposto depende do estado de origem */
switch (cod_estado) {
    case 1: aliquota = 0.35; break;
    case 2: aliquota = 0.25; break;
    case 3: aliquota = 0.15; break;
    case 4: aliquota = 0.05; break;
    default: aliquota = 0.00; /* estado 5: isento */
}

imposto = preco_carga * aliquota;
total = preco_carga + imposto;

printf("\nPeso em quilos...: %.2f kg\n", peso_kg);
printf("Preco da carga...: R$ %.2f\n", preco_carga);
printf("Valor do imposto.: R$ %.2f\n", imposto);
printf("Valor total.....: R$ %.2f\n", total);
return 0;
}

```

Teste você mesmo

Entrada	Saída esperada
Estado 1, 2 t, carga 25	2000 kg — R\$ 500.000,00 — imposto R\$ 175.000,00 — total R\$ 675.000,00
Estado 5, 1 t, carga 15	1000 kg — R\$ 100.000,00 — imposto R\$ 0,00 — total R\$ 100.000,00
Estado 3, 1 t, carga 20	Preço por quilo 100 (fronteira da primeira faixa)
Estado 3, 1 t, carga 21	Preço por quilo 250 (fronteira da segunda faixa)
Estado 2, 1 t, carga 31	Preço por quilo 340 — imposto de 25%

Apêndice — Compilando e testando

Compilação

No terminal do Linux ou do Mac, dentro da pasta onde está o arquivo:

```
gcc exercicio.c -o exercicio          # programas sem math.h
gcc exercicio.c -o exercicio -lm      # programas que usam math.h
./exercicio                          # executa
```

Vale a pena acrescentar as opções de aviso, que apontam problemas que o compilador aceitaria em silêncio:

```
gcc -Wall -Wextra exercicio.c -o exercicio -lm
```

Se o compilador reclamar de algo, leia a mensagem antes de mudar o código. Ela quase sempre indica a linha e o tipo do problema. Avisos não impedem o programa de rodar, mas costumam apontar exatamente o erro que faria o resultado sair errado.

Uma rotina de teste que funciona

1. Para cada tabela do enunciado, anote os valores de fronteira.
2. Para cada fronteira, teste o valor exato, um pouco abaixo e um pouco acima.
3. Teste também um valor bem no meio de cada faixa, para confirmar o caso comum.
4. Se o enunciado prevê entrada inválida, teste uma.
5. Confira cada resultado na mão, com calculadora. Um programa que roda sem travar não é o mesmo que um programa correto.

Uma última palavra

Se um exercício não sair de primeira, isso não é sinal de que você não entendeu a matéria. Programação se aprende errando em pequenas doses e corrigindo. O aluno que roda o programa, vê o resultado errado e descobre por quê aprende bem mais do que aquele que acerta de primeira copiando.